

Contents

PostgreSQL RLS, Transactions, and Tenant Context	1
What I learned	1
app.tenant_id vs tenant_id	2
Why the tenant context is transaction-scoped	2
Does every query become a transaction?	3
Why RLS is stronger than automatic ORM filtering	4
RLS does not mean one PostgreSQL role per SaaS user	4
Connection to transactions	4
The overall mental model	5
Key takeaway	6
Related concepts	6

PostgreSQL RLS, Transactions, and Tenant Context

Last updated: 2026-08-14

What I learned

In a shared-schema multi-tenant SaaS, every tenant-owned row can carry a `tenant_id`:

`courses`

id	name	tenant_id
1	Maths	VNR
2	DS	VNR
3	Physics	CBIT
4	Chemistry	CBIT

There are two broad ways to enforce tenant isolation:

Application / ORM filtering

The application or ORM adds the tenant condition to every query:

```
SELECT * FROM courses
WHERE tenant_id = 'VNR';
```

ORM middleware can automate this, so developers do not have to hand-write the filter each time. However, the enforcement still lives in the application layer. A raw SQL path, another service, or a bug/bypass in the middleware can potentially escape the protection.

PostgreSQL RLS

The application establishes the tenant context for the database transaction:

```
SET LOCAL app.tenant_id = 'VNR';
```

Then PostgreSQL's Row-Level Security policy enforces the boundary:

```
CREATE POLICY tenant_isolation ON courses
USING (tenant_id = current_setting('app.tenant_id')::text);
```

The application can now issue:

```
SELECT * FROM courses;
```

and PostgreSQL applies the tenant restriction through the RLS policy.

Mental model: application filtering says “remember to filter the tenant”; RLS says “tell the database which tenant this transaction represents, and the database enforces the boundary.”

app.tenant_id vs tenant_id

These are related but are **not the same thing**.

- `app.tenant_id` = transaction/request context established by the application: **“this transaction is acting on behalf of VNR.”**
- `courses.tenant_id` = data stored on the row: **“this course belongs to VNR.”**
- The RLS policy connects the two.

PostgreSQL does **not** inherently know that `Abhi` → `VNR`. The application authenticates `Abhi`, resolves his tenant, and establishes the tenant context. PostgreSQL then enforces the policy using that context.

`Abhi`

```
authentication / tenant lookup
↓
Abhi → VNR

SET LOCAL app.tenant_id = 'VNR'
↓
PostgreSQL transaction

↓
RLS policy

↓
row.tenant_id must match VNR
```

Why the tenant context is transaction-scoped

Suppose one HTTP request needs three database operations:

```
HTTP request
↓
BEGIN
↓
SET LOCAL tenant = VNR
↓
Query 1
↓
Query 2
```

↓
Query 3
↓
COMMIT

The tenant context is established **once for the transaction**, not before every query.

SET LOCAL is useful with connection pooling because the physical database connection does not permanently become a VNR connection. After the transaction ends, the connection can return to the pool and later serve another tenant.

Conceptually:

Connection #17

Request A
BEGIN
tenant = VNR
queries...
COMMIT
↓
connection returns to pool
↓
Request B
BEGIN
tenant = CBIT
queries...
COMMIT

Thousands of SaaS users therefore do **not** require thousands of database connections.

10,000 users
↓
HTTP requests
↓
A few API servers
↓
Connection pool
↓
A much smaller number of DB connections

Does every query become a transaction?

A single SQL statement executes within a database transaction even when the application does not explicitly issue BEGIN/COMMIT (an implicit transaction).

However, if tenant context is established with SET LOCAL, the context-setting statement and the protected query need to share an explicit transaction scope. An ORM can hide this transaction management from the application code.

So the useful mental model is:

RLS does not require a manually written transaction around every query. But if the RLS policy depends on transaction-local tenant context, the context and

the queries using it must live inside the same transaction.

Why RLS is stronger than automatic ORM filtering

An ORM middleware can make this:

```
SELECT * FROM courses WHERE tenant_id = 'VNR';
```

happen automatically. Functionally, that can look almost identical to RLS.

The difference is the **security boundary**:

ORM middleware

↓

application-level enforcement

↓

can potentially be bypassed by another data-access path

versus:

Application establishes tenant context

↓

PostgreSQL RLS

↓

database-level enforcement

↓

protected table operations are subject to the policy

A strong architecture can use both: ORM-level scoping for developer ergonomics and PostgreSQL RLS as the database-level backstop.

RLS does not mean one PostgreSQL role per SaaS user

A common misconception is that PostgreSQL needs a separate role for every application user.

It does not.

Application identity:

Abhi → VNR → Professor

PostgreSQL identity:

app_user

Thousands of application users can share one or a few PostgreSQL roles. Application authorization decides what Abhi is allowed to do; RLS can enforce which tenant rows the database operation can access.

Connection to transactions

Transactions are a general relational-database concept, not an RLS concept.

A transaction groups database operations into one logical unit of work. If a transaction is rolled back, changes made within it are undone. This is the **atomicity** part of ACID.

BEGIN

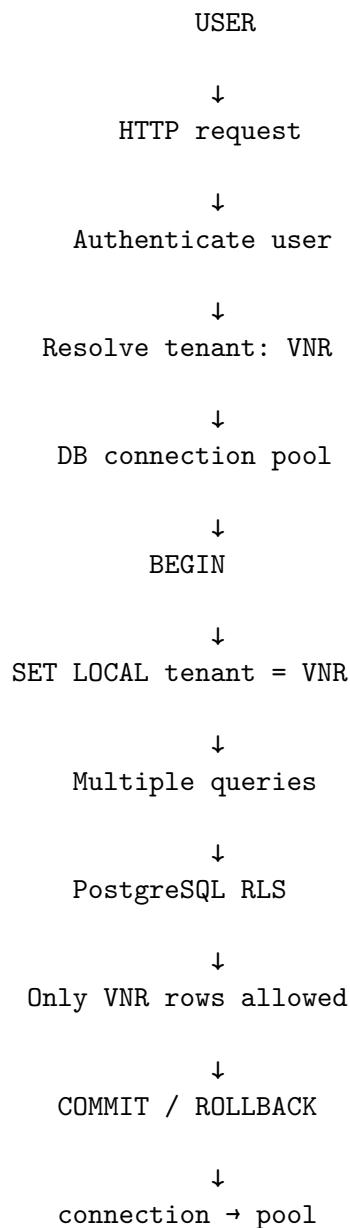
↓

Operation 1
↓
Operation 2
↓
Operation 3
↓
COMMIT → all changes persist

Failure + ROLLBACK → changes from the transaction are undone

RLS is simply using the transaction boundary for an additional purpose: safely scoping the tenant context.

The overall mental model



Key takeaway

The application still has to tell PostgreSQL which tenant the transaction represents. RLS does not discover the tenant; it enforces the tenant boundary once the application establishes that context.

The big architectural difference is **who guarantees the filter**:

- Application/ORM: *the application guarantees the tenant filter is added.*
 - RLS: *PostgreSQL guarantees the policy is applied to the protected table.*
-

Related concepts

- Multi-tenancy isolation models → `multi-tenancy.md`
- PostgreSQL schemas and tables
- Database transactions and ACID
- Connection pooling
- Authentication and tenant resolution
- Application authorization / RBAC
- ORM middleware and repository patterns